

Examen Final de GRAU-IA

(18 de junio de 2024)

Duración: 2 horas 30 minutos

1. (5 puntos) La manera habitual de poner precio a bienes cuyo valor no es objetivo es la tasación. Este tipo de valoración es la que se utiliza por ejemplo para las viviendas. Existen muchos sistemas de tasación, pero habitualmente se basan en un conjunto de características cualitativas que se obtienen de la valoración de las características de la vivienda y su ubicación.

En el caso de las viviendas se recopila información sobre el piso en el que está (bajos, entresuelo, primero, segundo...) la superficie total y la superficie habitable (ambas en m^2), el ancho y el largo de la planta de la vivienda (en m), si tiene certificado energético (A, B, C, D, E, F, G ó *ninguno*), si tiene calefacción (central, a gas, eléctrica), las frigorías del sistema de aire acondicionado (0 si no tiene), si tiene instalación de fibra óptica, la orientación de la mayoría de ventanas (norte, noreste, este, sureste...) el tiempo que lleva deshabitada (en semanas) y el total de impuestos y tasas que se han de pagar al año (en euros). Para cada una de las estancias de la vivienda (baño, cocina, habitación...) se guarda la superficie (en m^2), el material del suelo (madera, gres, acrílico), el material de las paredes (gres, pintura, papel pintado), la calidad de los acabados (superior, alta, media, baja), si hay aparato calefactor en la estancia, si hay aparato de aire acondicionado, la anchura de la puerta más estrecha en la estancia (en cm, para saber si puede pasar una silla de ruedas) y si necesita reforma. Aparte de esa información, en el caso del baño se guarda el tipo de ducha (bañera o plato de ducha), si tiene armarios y si hay bidet. En el caso de la cocina se guarda el tipo (cerrada, semicerrada en un segundo ambiente o abierta estilo americano), la superficie útil de trabajo (en cm^2), el material de las superficies de trabajo (marmol, madera, composite) y si tiene una isla central con los fogones. Del lavadero se guarda si es una estancia independiente o forma parte de la cocina. Del comedor se guarda si tiene una chimenea y el numero de columnas que tiene en su interior (0 si no hay). De cada habitación se guarda también si tiene armarios empotrados y si tiene baño propio. De los pasillos y distribuidores se guarda el largo, el ancho máximo y el ancho mínimo (los tres datos en metros). En los balcones se guarda el largo y el ancho (en metros) y la proporción de terraza que está cubierta por techo (en %). De trasteros y altillos se guarda el volumen total (en m^3).

En el caso de los edificios se guarda el año de construcción, la planta del edificio (en m^2), el tipo de construcción (vivienda unifamiliar aislada, vivienda adosada, edificio aislado o edificio adosado), la calle y el número, el código postal, el número de vecinos, el gasto comunitario mensual (en euros), si tiene portero o no, el ancho mínimo de las rampas de acceso (0 si no tiene), el tamaño del ascensor (largo y ancho dentro de la cabina), la altura total del edificio (en metros), y el año de la última reforma de fachada. De la calle se guarda también el tipo de via (avenida, calle, paseo, calle peatonal, Zona 30, ...), a que barrio pertenece, la anchura de aceras y la distancia mínima entre las fachadas de los edificios de lado a lado de la calle (ambos en metros). El edificio puede tener una serie de elementos comunitarios (placas solares, parking, trasteros, terraza comunitaria, piscina, gimnasio comunitario). De la piscina se guarda el largo y el ancho (en metros) y si tiene vestidores o taquillas, de las placas solares se guarda la potencia media de generación mensual (en vatios), del parking se guarda el número de plazas, de los trasteros su número y el tamaño medio (m^3) y de la terraza comunitaria su superficie (en m^2) y si los vecinos pueden colgar ropa allí.

De los barrios se guarda el nombre, el tipo de zona (centro, periferia, turística, industrial...), el precio medio del metro cuadrado de vivienda, el tiempo medio que se tarda en llegar al centro en transporte público (en minutos) y el tiempo medio que se tarda en salir desde allí a las vías rápidas fuera de la ciudad (en minutos). Cada barrio tiene asociado una serie de equipamientos/servicios (locales de ocio, comercios, centros de barrio, parques y jardines, centros de culto, farmacias, oficina de correos, comisaria de policía, hospitales, centros de atención primaria...). De todos ellos se guarda el nombre, la calle, el número y el horario de apertura (un string del estilo "lunes a viernes de 9 a 13 y de 15 a 18 horas" o "24/7" en el caso de que esté abierto las 24 horas del día los 7 días de la semana). De los locales de ocio queremos guardar el tipo (bares, restaurantes, discotecas, pubs musicales, salas de baile, salones de fiestas, ...) y el aforo (en número máximo de personas). De los centros de culto queremos saber la religión (un string) y el aforo máximo. De los centros educativos se guarda el tipo de estudios (pre-escolar, primaria, secundaria, universitarios, academia, ...). También se guarda un listado de las paradas de bus, metro, tranvia, tren y taxi dentro de cada barrio. Para todas ellas se tiene las coordenadas GPS (un string) y una descripción textual (string) de las líneas que pasan.

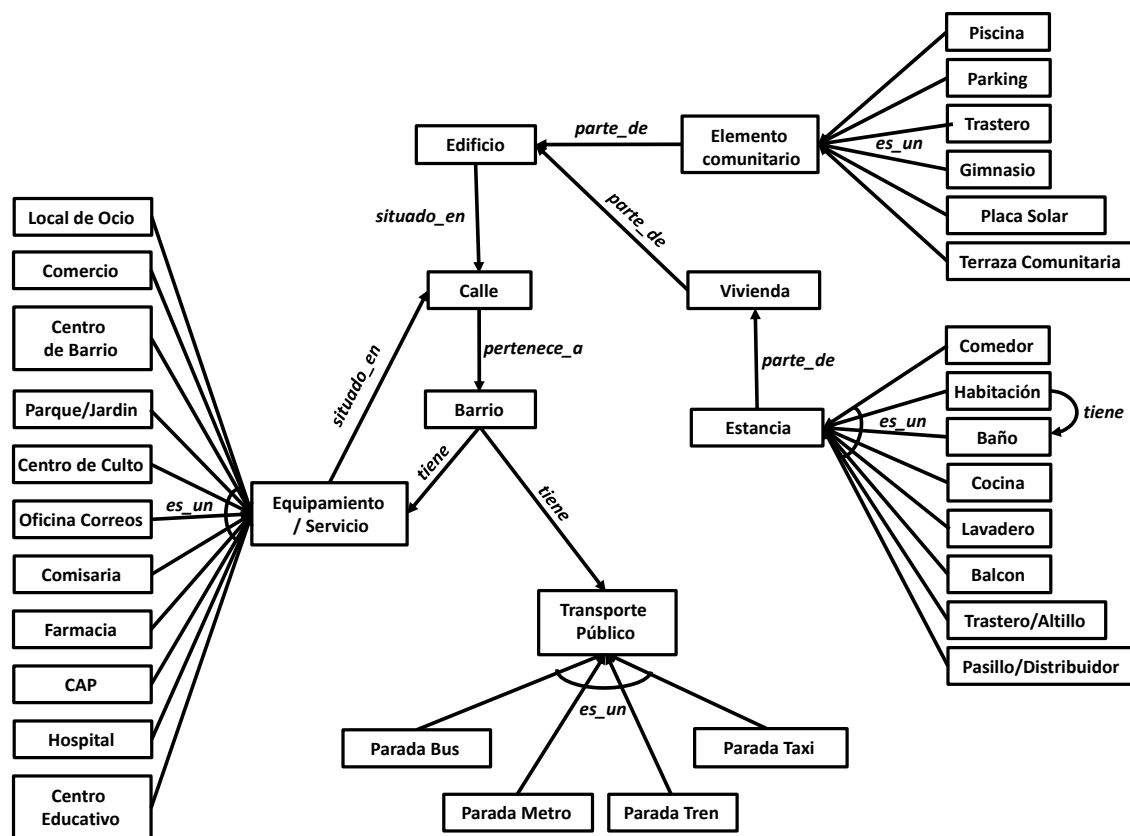
Por lo que nos dicen los expertos se suelen usar cuatro características para tasar una vivienda:

- El **entorno**, con una valoración de 1 a 5, que tendría en cuenta el número, tipo y proximidad de los equipamientos/servicios de la zona del inmueble (jardines, bibliotecas, centro de atención primaria, hospitales, colegios, áreas comerciales, ...), las comunicaciones de la zona (transporte público, vías rápidas, ...) y la zona en la que está ubicada (centro, periferia, zona turística, zona industrial...).
- La **calidad del edificio**, con una valoración de 1 a 5, que tendría en cuenta su antigüedad, el tipo de construcción, la superficie de su planta, su altura, la existencia de elementos de accesibilidad (rampas, barandillas, ascensor, anchura de la entrada, de los rellanos, ...), el número de vecinos, la anchura de la calle, la existencia de aparcamiento en el edificio, los gastos de la comunidad, ...
- La **calidad de la vivienda**, con una valoración de 1 a 5, que tendría en cuenta su superficie total, su superficie habitable, la relación entre la anchura y la longitud de la vivienda, si necesita reformas, la calidad de los acabados, el tiempo que lleva deshabitada, ...
- La **habitabilidad/funcionalidades** con una valoración de 1 a 5, que tendría en cuenta el número de habitaciones, el número de baños, la existencia de balcones y terrazas, elementos extra (suelo de madera, calefacción, trasteros, armarios empotrados, ...), ...

A partir de estas características queremos determinar si la vivienda se ha de tasar a precio de mercado, por debajo del precio de mercado o por encima del precio de mercado. El precio de mercado se obtiene a partir del precio medio del m² de la zona. Para determinar la variación respecto al precio de mercado, usaremos las características particulares de la vivienda para cuantificar la variación del precio. Por ejemplo, en el caso de que se clasifique la vivienda por debajo del precio de mercado, si la vivienda se ha de reformar el precio de mercado se rebajaría en un 20 %, o si hay pocos servicios en el entorno se rebajaría un 5 %. En el caso de que se clasifique la vivienda por encima del precio de mercado, si la vivienda tiene muchos elementos extra de habitabilidad como suelo de madera, o calefacción se le añadirán 3000 euros por cada elemento extra, o si el edificio tiene elementos de accesibilidad, como por ejemplo rampas, se añadirán 2000 euros al precio.

- a) (2,5 puntos) Diseña la ontología del dominio descrito, incluyendo todos los conceptos que aparecen en la descripción e identificando los atributos más relevantes. Lista que conceptos forman parte de los datos de entrada del problema y que conceptos forman parte de la solución. Justifica las decisiones de diseño que has realizado. En el caso de las subclases de una clase, justifica con un texto breve la necesidad de crear dichas subclases. Para cada concepto de la ontología lista todos sus atributos (para cada atributo se ha de indicar su nombre y su tipo, y en el caso de las enumeraciones, sus valores posibles). [Nota: tened en cuenta que la ontología puede necesitar modificaciones para adaptarla al apartado siguiente.]

En este problema la ontología ha de incorporar, como mínimo, todos los conceptos que forman parte de la entrada del problema: la información que se obtiene directamente de la base de datos y la información derivada. En este caso no hace falta incorporar conceptos de la solución, ya que solo es una valoración numérica de conceptos ya presentes en los datos de entrada (un atributo en la clase **Vivienda**).



De la ontología resultante cabe remarcar que existen varios conceptos que comparten atributos y/o relaciones, y que esto ha sido algo que se ha tenido muy en cuenta a la hora de crear super-clases con las características comunes, o para decidir entre crear subclases o solo añadir un atributo **tipo**. La regla general es que hemos creado subclases cuando una o varias tienen atributos diferentes y/o relaciones diferentes. Ese es el caso de las subclases de **Elemento Comunitario**, **Estancia** y **Equipamiento/Servicio**. No se siguió del todo ese criterio en el caso de las subclases de **Transporte Público**, que a pesar de que las subclases no tienen ni atributos ni relaciones diferenciadas se han añadido siguiendo el heurístico del diseño de ontologías que recomienda que ramas hermanas de la taxonomía tengan un nivel de granularidad similar (en este caso las ramas de los elementos que tiene un **Barrio**).

Atributos: a continuación se listan los atributos mínimos para representar la información mencionada explícitamente en el enunciado y la necesaria para el apartado b) del problema. Hay otros atributos que se deberían añadir (como la referencia catastral de la vivienda o el NIF de su propietario) si quisiéramos una ontología lo más general y reutilizable posible.

- **Edificio:** num_calle (entero), código_postal (entero), año_construcción (entero), año_última_reforma (entero), planta (m²), altura_total (m), tipo (enumeración: {unifamiliar, unifamiliar_adosado, edificio, edificio_adosado}), portero? (booleano), gasto_comunitario_mensual (euros), largo_ascensor (cm), ancho_ascensor (cm), min_ancho_rampas (cm), *Calidad_edificio* (enumeración: {1, 2, 3, 4, 5});
- **Vivienda:** piso (enumeración: {bajos, entresuelo, 1o., 2o., 3o., ...}), superficie_total (m²), superficie_habitable (m²), planta_ancho (m), planta_largo (m), certificado_energético (enumeración: {A, B, C, D, E, F, G, ninguno}), calefacción (enumeración: {ninguna, central, gas, eléctrica}), frigorías_AC (real), fibra_óptica? (booleano), orientación (enumeración: {N, NE, E, SE, S, SO, O, NO}), tiempo_desabitada (semanas), impuestos_año (euros), *Calidad_vivienda* (enumeración: {1, 2, 3, 4, 5}), *Habitabilidad/Funcionalidad* (enumeración: {1, 2, 3, 4, 5}), *TASACIÓN* (enumeración: {precio_mercado, por_debajo_mercado, por_encima_mercado}), VALOR_TASACIÓN (euros);

- **Estancia:** superficie (m^2), material_suelo (enumeración: {madera, gres, acrílico}), material_paredes (enumeración: {gres, pintura, papel}), calidad_acabados (enumeración: {superior, alta, media, baja}), calefactor? (booleano), AC? (booleano), min_ancho_puerta (cm), necesita_reforma? (bool);
- **Comedor:** chimenea? (booleano), num_columnas (entero);
- **Habitación:** armarios_empotrados? (booleano);
- **Baño:** tipo_ducha (enumeración: {bañera, plato}), armarios? (booleano), bidet? (booleano);
- **Cocina:** tipo (enumeración: {cerrada, semicerrada, abierta}), superficie_útil (cm^2), material_superficie (enumeración: {marmol, madera, composite}) isla_central? (booleano);
- **Lavadero:** independiente? (booleano);
- **Balcón:** largo (m), ancho (m), cubierta (%);
- **Trastero/Altillo:** vol_total (m^3);
- **Pasillo/Distribuidor:** largo (m), max_ancho (m), min_ancho (m);
- **Piscina:** largo (m), ancho (m), vestidores? (booleano), taquillas? (booleano);
- **Parking:** num_plazas (entero);
- **Placa Solar:** potencia_media_mes (watt);
- **Terraza Comunitaria:** superficie (m^2), tender_ropa? (booleano);
- **Calle:** tipo_via (enumeración: {avenida, calle, paseo, peatonal, zona_30, ... }), ancho_acera (m), distancia_fachadas (m);
- **Barrio:** nombre (string), tipo (enumeración: {centro, periferia, turística, industrial}), precio_medio_m2 (euros), tiempo_al_centro (minutos), tiempo_salir (minutos), Entorno (enumeración: {1, 2, 3, 4, 5});
- **Equipamiento/Servicio:** nombre (string), núm_calle (entero), horario (string);
- **Transporte Público:** coordenadas_GPS (string), líneas (string);
- **Local Ocio:** tipo (enumeración: {bar, restaurant, discoteca, pub_musical, sala_baile, salón_fiestas, ... }), aforo (int);
- **Centro de Culto:** religión (string), aforo (int);
- **Centro Educativo:** tipo_estudios (enumeración: {preescolar, primaria, secundaria, universitaria, academia, ... });

Como se puede ver, hemos decidido también representar las características abstractas del problema, las de la solución abstracta y la solución concreta (son los que tienen el nombre en *Cursiva*, *CURSIVA* o *MAYUSCULAS*, respectivamente). De esta manera todos los conceptos que aparecerán en las reglas están soportados por la ontología.

- b) (2,5 puntos) El problema descrito es un problema de análisis. Explica cómo lo resolverías usando clasificación heurística. Justifica si el tipo de solución que se pide requiere de refinamiento o no. Lista las variables que forman parte del Problema Abstracto (nombre y valores posibles). Lista las variables que forman parte de la Solución Abstracta (nombre y valores posibles). Da al menos 3 ejemplos de reglas (suficientemente variadas, utilizando diferentes conceptos) para cada una de las fases de esta metodología, usando los conceptos de la ontología desarrollada en el apartado anterior. Para las reglas podéis usar la notación de alto nivel que usamos en los ejercicios de problemas (si ... entonces ...) o una notación más próxima a la del lenguaje CLIPS (siempre que queden claro los elementos del antecedente de la regla y del consecuente).

Para resolverlo mediante clasificación heurística debemos identificar en el problema las diferentes fases y elementos de esta metodología. En este caso hay solo una opción posible: la solución que pide el enunciado solo puede ser una solución concreta, ya que es imposible poder calcular el valor final de tasación en el nivel de solución abstracta sin acceder a los datos concretos como los m^2 de la vivienda o el precio medio del m^2 en el barrio.

Una vez sabemos cuantas fases requiere nuestra solución ya podemos enunciar la solución completa. El primer elemento son los problemas concretos que hay que tratar. En este dominio los problemas concretos están definidos por toda la información detallada sobre la vivienda, el edificio y su entorno que tenemos identificada en la ontología. El segundo elemento son los problemas abstractos, estos estarán definidos a partir de las cuatro características que menciona el enunciado (*Entorno*, *Calidad_edificio*, *Calidad_vivienda* y *Habitabilidad/Funcionalidad*), todas ellas con un rango de

valores discretos de 1 a 5.

Para conectar los problemas concretos con los abstractos necesitamos definir las reglas de abstracción de datos, por ejemplo:

- si Vivienda.superficie_habitable > 160m² y Vivienda.tiempo_deshabitada < 26 semanas y $0.8 \leq (\text{Vivienda.planta_largo} \div \text{Vivienda.planta_ancho}) \leq 1.2$ y cardinalidad(Estancia.necesita_reforma==true)==0 y cardinalidad(Estancia.calidad_acabado==superior) $\div$ cardinalidad(Estancia) > 0.7 entonces Vivienda.Calidad_vivienda=5;
- si Vivienda.piso == (Bajos o Entresuelo) y cardinalidad(Habitación.parte_de.Vivienda)<3 y cardinalidad(Habitación.armarios_empotrados==true)<2 y cardinalidad(Baño.parte_de.Vivienda)<2 entonces Vivienda.Habitabilidad/Funcionalidad=1;
- si Barrio.tipo==periferia y Barrio.tiempo_al_centro > 45min y Barrio.tiempo_salir > 30min y cardinalidad(Barrio.tiene.Comisaria)==0 y cardinalidad(Barrio.tiene.Comercio)==0 y cardinalidad(Barrio.tiene.Parque/Jardín)< 2 entonces Barrio.Entorno=1;
- si Edificio.año_construcción<1976 y Edificio.año_última_reforma<2006 y $(\text{Edificio.planta_largo} * \text{Edificio.planta_ancho}) < 100\text{m}^2$ y $(\text{Edificio.largo_ascensor} * \text{Edificio.ancho_ascensor}) < 1\text{m}^2$ entonces Edificio.Calidad_edificio=1;

El tercer elemento son las soluciones abstractas. En este caso tenemos solo tres soluciones abstractas: tasar a precio de mercado, tasar por debajo del precio de mercado y tasar por encima del precio de mercado.

Para ligar los problemas abstractos con las soluciones abstractas necesitaremos reglas de asociación heurística, como por ejemplo:

- si (Barrio.Entorno == 3 o Barrio.Entorno == 4) y Edificio.Calidad_edificio==3 y Vivienda.Calidad_vivienda==3 y Vivienda.Habitabilidad/Funcionalidad==3 entonces Vivienda.TASACIÓN=Tasar_a_precio_mercado;
- si (Barrio.Entorno == 4 o Barrio.Entorno == 5) y Edificio.Calidad_edificio==4 y Vivienda.Calidad_vivienda==5 y Vivienda.Habitabilidad/Funcionalidad==5 entonces Vivienda.TASACIÓN=Tasar_por_encima;
- si Barrio.Entorno < 3 y (Edificio.Calidad_edificio<3 o Vivienda.Calidad_vivienda<3 o Vivienda.Habitabilidad/Funcionalidad==3) entonces Vivienda.TASACIÓN=Tasar_por_debajo;

El cuarto y último elemento son las soluciones concretas. En este caso corresponde al cálculo del valor concreto de tasación de la vivienda, echando mano de la solución abstracta y de los datos concretos del problema. Lo que sigue son algunos ejemplos de reglas:

- PrecioMercado = Barrio.precio_medio_m2 * Vivienda.superficie_total;
- si Vivienda.TASACIÓN==Tasar_a_precio_mercado entonces Vivienda.VALOR_TASACIÓN=PrecioMercado;
- si Vivienda.TASACIÓN=Tasar_por_debajo y cardinalidad(Habitación.necesita_reformas==true) > 1 entonces Vivienda.VALOR_TASACIÓN=PrecioMercado*0.8;
- si Vivienda.TASACIÓN=Tasar_por_encima entonces para cada E.parte_de.Vivienda tal que elementoExtra(E) : incrementar Vivienda.VALOR_TASACIÓN en 3000 euros para cada E.parte_de.Edificio tal que elementoAccesibilidad(E) : incrementar Vivienda.VALOR_TASACIÓN en 2000 euros

2. (5 puntos) El mago Rincewind debe ayudar con el diseño cognitivo de un Golem mensajero. Un Golem es un artillugio que se alimenta de una batería arcana (que tiene una carga) para desplazarse y hacer

diversas tareas. Esta se consume a un ritmo dependiente de las acciones que el Golem toma: por ejemplo, desplazarse 100 metros (tamaño exacto de un desplazamiento estándar) son 20 unidades de carga, entregar un mensaje depende de la longitud de este, etc. El apartado *b)* determina los costes para el modelo actual diseñado por el departamento arcanomecánico. Además, el Golem dispone de una cantidad de oro que puede usar para recargar su batería (1 oro recarga la batería a su máximo independientemente de la carga actual) en localizaciones específicas.

Además, un Golem es especialmente lento en desplazamiento (por lo que Rincewind considera este proyecto un gasto terrible de fondos públicos). Por ello, Rincewind ha decidido también otorgar al golem la capacidad de pagar por el uso de carruajes para el desplazamiento. Los carruajes tienen un coste (en oro) asociado a sus posibles desplazamientos. Los carruajes solo pueden desplazarse por carreteras, así que no se pueden utilizar, por ejemplo, dentro de un edificio. Su accesibilidad viene determinada como parte del problema especificado. Para usar un carruaje, se debe llamar y subir a uno, desplazarse y pagar y finalmente bajar en tu destino.

El objetivo del diseño cognitivo del que Rincewind está encargado es determinar la secuencia de acciones a realizar para que el Golem entregue su mensaje al destinatario. El problema viene descrito como un mensaje (con un coste en batería por entregar), una dirección, batería (con una carga máxima determinada) totalmente cargada, oro inicial y un mapa con información sobre rutas de carruajes y accesibilidad entre calles. En el plan del Golem deberá respetar las restricciones sobre oro y batería (como no pagar más oro del que tiene, o no hacer acciones cuyo coste en batería exceda su batería actual). El plan del Golem debería detallar los desplazamientos que hace a pie (de localización en localización), subir a un carruaje, desplazarse en carruaje, bajar de un carruaje, cargar su batería en un punto de carga, y la entrega del mensaje final.

- a)* (4 puntos) Describe el dominio (incluyendo predicados, acciones, etc.) usando PDDL. Tened en cuenta que los detalles del apartado *b)* dependen de la implementación del Golem y el dominio debe poder adaptarse a cambios sobre esta.

Se ha optado por modelar el dinero de recarga de forma diferente como ejemplo, ya que no se pedía adaptabilidad sobre ello. Para comprobar si un carruaje puede ir a un sitio, sencillamente se definirá el fluente como -1 en recorridos imposibles.

```
(define (domain Golem)
  (:requirements :adl :fluents)
  (:predicates
    (at ?pl) (accessible ?pl1 ?pl2)
    (on_carriage) (carriage_at ?pl) (can_board_carriage ?pl)
    (has_charger ?pl) (final_destination ?pl) (done)
  )
  (:functions
    (gold) (cost_carriage ?pl1 ?pl2)
    (current_battery) (max_battery)
    (cost_msg) (cost_move)
    (cost_board) (cost_unboard)
  )
  (:action walk
    :parameters (?orig ?dest)
    :precondition (and (at ?orig) (accessible ?dest)
                       (>= (current_battery) (cost_move)))
    :effects (and (not (at ?orig)) (at ?dest)
                  (decrease (current_battery) (cost_move)))
  )
  (:action board_carriage
    :parameters (?place)
```

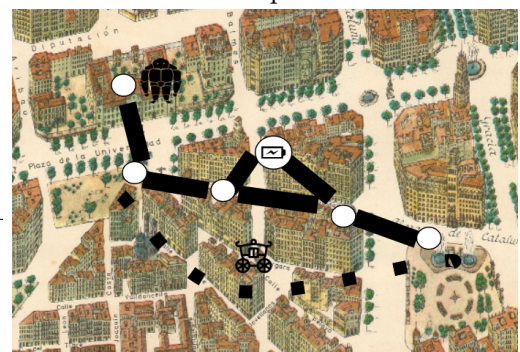
```

:precondition (and (at ?place) (can_board_carriage ?place)
                  (>= (current_battery) (cost_board)))
:effects (and (not (at ?place)) (on_carriage) (carriage_at ?place)
             (decrease (current_battery) (cost_board)))
)
(:action travel_by_carriage
 :parameters (?orig ?dest)
 :precondition (and (on_carriage) (carriage_at ?place)
                   (< 0 (cost_carriage ?orig ?dest)))
                   (>= (gold) (cost_carriage ?orig ?dest)))
 :effects (and (not (carriage_at ?orig)) (carriage_at ?dest)
              (decrease (gold) (cost_carriage ?orig ?dest)))
)
(:action unboard_carriage
 :parameters (?place)
 :precondition (and (on_carriage) (carriage_at ?place)
                   (>= (current_battery) (cost_unboard)))
 :effects (and (not (on_carriage)) (at ?place)
              (decrease (current_battery) (cost_unboard)))
)
(:action recharge
 :parameters (?place)
 :precondition (and (at ?place) (has_charger ?place)
                   (>= (gold) 1))
 :effects (and (assign (current_battery) (max_battery))
              (decrease (gold) 1))
)
(:action speak_msg
 :parameters (?place)
 :precondition (and (at ?place) (final_destination ?place)
                   ((>= (current_battery) (cost_msg))))
 :effects (and (decrease (current_battery) (cost_msg)) (done))
)
)
)

```

- b) (1 puntos) Representa el fichero de problema para la siguiente especificación de Golem (Versión A), teniendo que entregar el mensaje 'Ook. Ook.' desde la Biblioteca de la UI, a Plaza Tarraconensis. Cada desplazamiento descrito en la 2ª tabla es de exactamente 100 metros, las entradas en cursiva indican un lugar de recarga. La imagen es una representación visual del mapa:

VA-Golem	Unidades
Carga Máxima	100 E
Oro inicial	20 o
Coste mensaje	10 E
Coste Subir Carruaje	3 E
Coste Bajar Carruaje	2 E
Carruaje Origen	Destino (coste)
Universidad Invisible	Plaza Tarraconensis (10 o)
Plaza Tarraconensis	Universidad Invisible (10 o)



Lugar Origen	Accesible Andando
Biblioteca UI	Universidad Invisible
Universidad Invisible	Biblioteca UI, Ronda Universidad 1
Ronda Universidad 1	Universidad Invisible, Ronda Universidad 2, Puesto de carga
<i>Puesto de carga</i>	Ronda Universidad 1, Ronda Universidad 2
Ronda Universidad 2	Ronda Universidad 1, Puesto de carga, Plaza Tarraconensis
Plaza Tarraconensis	Ronda Universidad 2

```

(define (problem 5Nexus)
  (:domain RincewindNNexus)
  (:objects
    Biblio UI Ronda1 Ronda2 PuestoCarga Plaza
  )
  (:init
    (at Biblio)
    (accesible Biblio UI) (accesible UI Biblio)
    (accesible UI Ronda1) (accesible Ronda1 UI)
    ...

    (can_board_carriage UI) (can_board_carriage Plaza)
    (has_charger PuestoCarga)
    (final_destination Plaza)

    (= (current_battery) 100) (= (max_battery) 100)
    (= (gold) 20)
    (= (cost_msg) 10) (= (cost_move) 20)
    (= (cost_board) 3) (= (cost_unboard) 2)

    (= (cost_carriage UI Plaza) 10) (= (cost_carriage Plaza UI) 10)
    ; Para el resto de fluentes, definir a 0 es default (pero lo explicitamos)
    (= (cost_carriage Biblio UI) 0) (= (cost_carriage Biblio Ronda1) 0) ...
  )
  (:goal (done))
)

```

Las notas se publicaran el día **26 de junio**.